

CMPT 383 Quiz 1 (Python)

Summer 2026

Duration	30 minutes for the entire exam.
Coding Questions	For coding questions, use pseudocode in the style of the language the question is about.
Aids allowed	Pencil or pen, and eraser. No notes, no papers, no books, no computers, no calculators, no cheat sheets, etc.
During the exam	Raise your hand if you would like to speak with a proctor. To be fair to all students, we don't answer questions about the exam. Choose the answer based on your best understanding of the question. Re-reading the question may help.

Python Quiz

1. Write a function called `div_pairs(nums)` that takes a list of positive integers `nums` and uses a single list comprehension to return all pairs `(a, b)` from the list such that `a` is less than `b` and `b` is a multiple of `a`. You can assume `nums` is non-empty and has no duplicates.

For example, `div_pairs([6, 2, 3, 4, 7])` returns `[(2, 6), (2, 4), (3, 6)]`.

Solution

```
def div_pairs(nums):  
    return [(a, b)  
            for a in nums  
            for b in nums  
            if a < b  
            if b % a == 0  
            ]
```

2. Write a Python **generator function** (using `yield`) called `gen_non_vowels(s)` that iterates through the characters of string `s`, but skips vowels. Assume the vowels are the five lowercase letters `a`, `e`, `i`, `o`, and `u`.

For example:

```
for c in gen_non_vowels('rate!'):
    print(c)
```

Prints:

```
r
t
!
```

Solution

```
def gen_non_vowels(s):
    for c in s:
        if c not in 'aeiou':
            yield c
```

3. Write a Python function called **make_greet_fn(greeting)** that returns a greeting function that prints a greeting for a given name. It also prints the number of times the greeting function has been called. For example:

```
say_hi = make_greet_fn('Hiya')
say_hi('Alice')    # 1 Hiya Alice!
say_hi('Bob')      # 2 Hiya Bob!
say_hi('Charlie')  # 3 Hiya Charlie!

say_hello = make_greet_fn('Hello')
say_hello('Alice') # 1 Hello Alice!
say_hello('Bob')   # 2 Hello Bob!
say_hello('Charlie') # 3 Hello Charlie!
say_hello('Dana')  # 4 Hello Dana!
```

The number that is printed is how times function has been called: every time it is called the number should be increased by 1.

Solution

```
def make_greet_fn(greeting):
    c = 0
    def greet(name):
        nonlocal c
        c += 1
        print(c, f"{greeting} {name}!")
    return greet
```